

Dynamic Fluid Caustics

Interactive Wave Control & Inverse Rendering Simulator

A real-time interactive physics sandbox that converts a user-sketched 2D image into transient liquid-surface geometry whose refracted caustic reconstructs that image on the tank floor. This document refines the source outline into a low-level engineering specification covering component architecture, the inverse-caustic and wave-solver mathematics, memory and GPU acceleration strategy, and implementation pseudocode.

DOCUMENT TYPE Low-Level Technical Specification	REVISION 2.0 — Expanded Architecture
TARGET PLATFORM Desktop GPU (Vulkan / OpenGL 4.5+ / WebGPU)	CORE LANGUAGES C++17 / Rust · GLSL / WGSL compute
SIMULATION REGIME Linearised shallow-water surface waves	REAL-TIME BUDGET ≤ 16.6 ms / frame (60 Hz)

Contents

1	Executive Summary
2	System Overview & Computational Pipeline
3	Component-Level Architecture Design
4	Mathematical & Algorithmic Specifics
5	Data Constraints & Acceleration Strategies
6	Wave-Physics Modelling Constraints
7	Pseudocode Blueprints
8	Development Milestones & Implementation Plan
9	Technical Challenges & Risk Mitigation
A	Nomenclature

1 Executive Summary

The system implements a four-stage inverse-rendering pipeline. A user sketches a 2D shape on a canvas; the software converts that sketch into a normalised light-density map $I(x,y)$, computes the refractive liquid surface $h(x,y)$ that would focus uniform incoming light into that density, and then determines the boundary

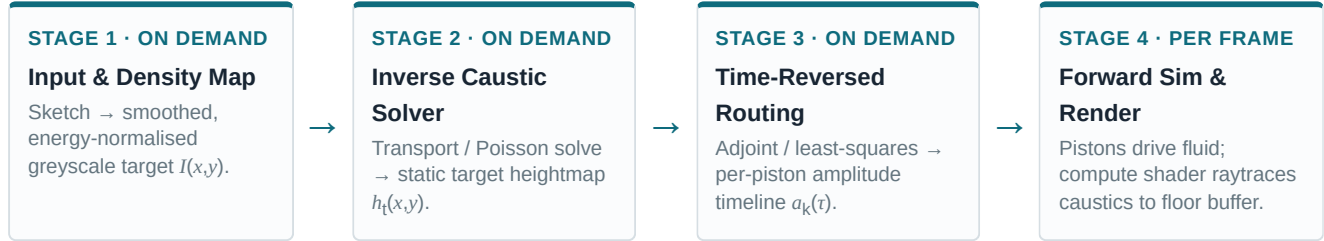
actuators (virtual pistons) whose interfering wavefronts momentarily reproduce that surface. A forward fluid simulation executes those actuators and a GPU caustic raytracer projects the resulting refraction onto the tank floor.

The central engineering claim is that the entire inverse problem can be made real-time by exploiting **linearity** at two independent points: (i) the inverse-caustic step is reduced from a nonlinear Monge–Ampère optimal-transport solve to a single **Poisson solve** under the paraxial approximation, and (ii) the piston-actuation step is reduced from a nonlinear control problem to a **linear least-squares projection** onto a pre-computed wave-basis matrix. Both reductions convert per-interaction cost from seconds to milliseconds, at the price of a bounded, characterisable approximation error. The remainder of this document specifies the modules, data contracts, discretisations, and acceleration structures required to realise this.

SCOPE. This revision assumes small-amplitude (linear) surface waves. Nonlinear breaking, wetting/contact-line dynamics, and full 3D Navier–Stokes are explicitly out of scope; they are neither required for the caustic effect nor compatible with the real-time budget.

2 System Overview & Computational Pipeline

The pipeline is a strictly sequential dataflow. Stages 1–3 run *on demand* (only when the user commits a new sketch); stage 4 runs *every frame*. This split is deliberate: the expensive inverse mathematics is amortised across the many frames of the forward playback.



2.1 Stage responsibilities

Stage 1 — Input & Density Mapping. A 2D canvas accepts strokes, text, or primitives. Raw input is rasterised, Gaussian-blurred to remove spatial frequencies the fluid cannot physically form, and normalised so total radiant energy is conserved (the mean of I equals the uniform source level). Output is a single-channel float image.

Stage 2 — Inverse Caustic Surface Generation. Given I , compute how uniform incoming rays must be redistributed to match it (optimal transport), convert the transport map into surface normals via Snell's law, and integrate to a heightmap h_t . Detailed in §4.1.

Stage 3 — Inverse Wave Actuation Mapping. Inject h_t into the wave solver and evolve it *backward* in time; energy reaching the tank perimeter is recorded as the required piston timeline. Equivalent (and numerically preferable) is the least-squares formulation of §4.3.

Stage 4 — Forward Simulation & Rendering. Pistons replay the recorded timeline in forward time; inbound wavefronts constructively interfere at the focal instant; a compute shader traces refracted light rays through the dynamic surface mesh and accumulates the caustic on the floor plane. Detailed in §5.2 and §7.

3 Component-Level Architecture Design

The four pipeline stages are realised by seven decoupled software modules plus an orchestrating state machine. Modules communicate only through the explicit data contracts of §3.2; no module reaches into another's internal state. This boundary makes the on-demand solver stack (M2–M4) independently testable against golden heightmaps, and lets the per-frame path (M5–M6) run on a separate cadence.

3.1 Module decomposition

Module	Stage	Responsibility	Key internal state
M1 · Input Processing Canvas	1	Capture pointer strokes/primitives; rasterise to a bitmap; expose "commit" event.	Stroke list, RGBA8 canvas texture, dirty flag.
M2 · Density Map Preprocessor	1	Greyscale conversion, separable Gaussian blur, band-limit clamp, energy normalisation.	Blur kernel radius σ , source level I .
M3 · Inverse Caustic Solver	2	Optimal-transport (Monge–Ampère) or Poisson solve; normal integration to heightmap.	Transport potential ϕ / u , solver tolerances, focal length d .
M4 · Wave Actuation Mapper	3	Project target heightmap onto pre-computed wave basis; emit per-piston timeline.	Basis matrix M (or its pseudoinverse), regulariser λ .
M5 · Fluid Simulation Engine	4	Advance damped wave field by FDM; apply piston boundary forcing; expose surface grid.	Ping-pong height buffers h^n, h^{n-1} ; SoA arrays.
M6 · GPU Render Pipeline	4	Build surface normals; raytrace/splat refraction; tone-map caustic; composite scene.	Floor accumulation buffer, normal texture, camera.
M7 · Orchestration State Machine	all	Sequence on-demand solves, drive playback clock, manage Pulse-Mode looping.	State $\in \{\text{IDLE, SOLVING, PLAYBACK, PULSE}\}$, playback cursor t .

3.2 Inter-module data contracts

Every arrow in the pipeline carries exactly one typed payload. The following contracts fix the structure, element type, and (where GPU-resident) texture format. Grid resolution $N = W \times H$ is the simulation grid; the solver grid may be down-sampled to $N' \leq N$ (see §5).

M1 → M2 · RawCanvas

```
struct RawCanvas {
    uint32_t width, height;    // canvas pixels
    Texture  bitmap;           // RGBA8, sRGB, GPU-resident
    AABB     ink_bounds;       // tight bbox of drawn content
};
```

M2 → M3 · DensityMap

```
struct DensityMap {
    uint32_t    Np;                // solver grid dim (N' per side)
    float*      I;                // row-major, length Np*Np, single channel
    float       I_bar;            // uniform source level = mean(I) > 0
    // GPU mirror: R32F texture, CLAMP_TO_EDGE
};
// Invariant: I[i] >= EPS > 0 (strictly positive; no zero density)
// Invariant: sum(I) == Np*Np * I_bar (energy conserved)
```

M3 → M4 · TargetHeightmap

```
struct TargetHeightmap {
    uint32_t    Np;
    float*      h_t;              // target surface height, metres, row-major
    float       d;               // focal distance surface→floor
    float       n_rel;           // relative refractive index n2/n1
};
```

M4 → M5 · PistonSchedule

```
struct PistonSchedule {
    uint32_t    num_pistons;      // P, indexed around tank perimeter
    uint32_t    num_steps;       // T_r reversed-time samples
    float*      a;               // amplitude[P][T_r], row-major per piston
    float       dt;              // timeline sample period (== sim dt)
    float       focal_time;      // t at which convergence occurs
};
```

M5 → M6 · FluidState (GPU-resident)

```
struct FluidState {
    uint32_t    W, H;
    Texture     height;          // R32F    – current surface h^n
    Texture     normal;          // RGBA16F – derived surface normals (xyz) + slope mag
    float       cell_size;      // dx (== dy), metres
};
```

```
struct CausticBuffer {  
    Texture    floor_accum;           // R32UI (fixed-point) during accumulation,  
                                       // resolved to R16F irradiance after tone pass  
    float      exposure;             // tone-map scale, auto or user-set  
};
```

DESIGN. The two on-demand outputs (TargetHeightmap, PistonSchedule) are plain CPU float arrays — small, cache-friendly, and trivially serialisable for golden-image regression tests. Only the per-frame payloads (FluidState, CausticBuffer) live on the GPU, minimising CPU ↔ GPU transfer to a single normal-texture write per frame.

4 Mathematical & Algorithmic Specifics

4.1 The inverse-caustic step

The problem: uniform incoming irradiance illuminates a refractive surface; find the surface that redistributes that light so the receiver (tank floor) shows target irradiance $I(x,y)$. This is a redistribution — an optimal-transport — problem. We give the exact formulation first, then the linearised approximation used at runtime.

4.1.1 Exact formulation — Optimal Transport / Monge–Ampère

Let μ be the (uniform) source measure over the surface domain Ω and ν the target measure with density I over the receiver domain Ω' . By Brenier's theorem the L^2 -optimal transport map is the gradient of a convex potential, $T = \nabla\phi$, and mass conservation $\nu(\nabla\phi(x)) \cdot \det(D^2\phi) = \mu(x)$ yields the **Monge–Ampère equation**:

$$\det(D^2\phi(x)) = \mu(x) / \nu(\nabla\phi(x)) \quad (4.1)$$

subject to the *second boundary condition* (transport BC): $\nabla\phi$ maps Ω onto Ω' , i.e. $\nabla\phi(\partial\Omega) = \partial\Omega'$. Equation (4.1) is a fully nonlinear elliptic PDE. It is solved by a damped Newton iteration on a finite-difference discretisation that preserves the discrete convexity of ϕ (a monotone / wide-stencil scheme is required for convergence). Each Newton step solves a sparse linear system of size N^2 .

Given ϕ , the transport map $T = \nabla\phi$ tells us each incoming ray's required landing point. The surface normal that bends a vertical ray at x toward $T(x)$ follows from **Snell's law** $n_1 \sin \theta_1 = n_2 \sin \theta_2$; integrating the resulting normal field (a screened-Poisson / least-squares normal integration) gives the exact heightmap h_t . Fidelity is high but the Newton solve costs tens of iterations of sparse linear algebra — *seconds* at interactive resolutions, too slow to run on every sketch edit.

4.1.2 Real-time approximation — Poisson (paraxial linearisation)

When deflections are small (surface near-flat, floor close relative to feature size), write the transport map as a small displacement of identity, $T(x) = x + \nabla u(x)$, with $|\nabla u| \ll 1$. The Jacobian is $J = I + D^2u$, and to first order $\det J \approx 1 + \text{tr}(D^2u) = 1 + \nabla^2 u$. Substituting into mass conservation $I = \mu / \det J$ and linearising collapses Monge–Ampère (4.1) into a single linear **Poisson equation**:

$$\nabla^2 u(x,y) = 1 - I(x,y) / \Gamma \quad (4.2)$$

solved with homogeneous Neumann boundary conditions (no transport across the tank wall). Because the operator is the constant-coefficient Laplacian on a regular grid, (4.2) is solved in $\mathbf{O}(N^2 \log N^2)$ by an FFT/DCT Poisson solver, or in $\mathbf{O}(N^2)$ by geometric multigrid — sub-millisecond at 256^2 . A further simplification is available: in the same paraxial regime the surface slope needed to steer a ray by $\nabla u/d$ is, by the thin-refractor form of Snell's law, $(n_{\text{rel}} - 1)\nabla h_t = -\nabla u/d$. Hence the heightmap is *directly proportional to the transport potential*:

$$h_t(x,y) = -u(x,y) / [d (n_{\text{rel}} - 1)] + \text{const} \quad (4.3)$$

so no separate normal-integration pass is required — one Poisson solve yields the target surface outright. This is the runtime path.

Property	Monge–Ampère / OT (exact)	Poisson (paraxial)
Governing PDE	$\det(D^2\phi) = \mu/\nu$ (nonlinear elliptic)	$\nabla^2 u = 1 - I/I$ (linear)
Solver	Damped Newton + monotone FD, sparse solve per step	FFT/DCT or multigrid, single solve
Cost	$O(N')$ per Newton step $\times$ tens of steps	$O(N' \log N')$ or $O(N')$
Wall time @256²	$\sim 10^2$ – 10^3 ms	< 1 ms
Valid for	Large redistribution, high contrast	Small deflection, moderate contrast
Error mode	Exact (to discretisation)	$O(\nabla u ^2)$ intensity error; softens high-contrast edges
Use	Offline / "high-quality" bake	Interactive default

STRATEGY. Ship Poisson as the interactive default and expose Monge–Ampère as an optional "refine" pass. Because both produce the same TargetHeightmap contract, M4 downstream is agnostic to which was used.

4.2 Forward fluid simulation — discretising the 2D shallow-water wave

The surface obeys the damped linear wave equation (the shallow-water limit for small amplitude):

$$\partial^2 h / \partial t^2 = c^2 \nabla^2 h - \gamma \partial h / \partial t \quad (4.4)$$

with wave speed $c = \sqrt{g \cdot H}$ (gravity $\times$ still depth) and damping γ . Discretise on a uniform grid with spacing $\Delta x = \Delta y$ using the **Finite Difference Method**. The spatial Laplacian uses the second-order five-point stencil:

$$L[h]_{i,j} = (h_{i+1,j} + h_{i-1,j} + h_{i,j+1} + h_{i,j-1} - 4h_{i,j}) / \Delta x^2 \quad (4.5)$$

Time is integrated with a central (leapfrog) second difference for the acceleration term and a centred first difference for the damping term:

$$(h^{n+1} - 2h^n + h^{n-1}) / \Delta t^2 = c^2 L[h^n] - \gamma (h^{n+1} - h^{n-1}) / (2\Delta t) \quad (4.6)$$

Solving (4.6) explicitly for the only future-time term h^{n+1} gives the update rule actually coded in M5:

$$h^{n+1} = [2h^n + (\frac{1}{2}\gamma\Delta t - 1)h^{n-1} + c^2\Delta t^2 L[h^n]] / (1 + \frac{1}{2}\gamma\Delta t) \quad (4.7)$$

This is a two-buffer scheme: it needs only h^n and h^{n-1} to produce h^{n+1} , so a ping-pong of two grids suffices (§5.1). Explicit integration is stable only under the 2D **Courant–Friedrichs–Lewy condition**:

$$c \Delta t / \Delta x \leq 1/\sqrt{2} \iff \Delta t \leq \Delta x / (c\sqrt{2}) \quad (4.8)$$

Δt is chosen at $\sim 0.9 \times$ this bound for margin. Two boundary treatments are needed: **piston edges** (Dirichlet forcing — the wall cell height is *set* from `PistonSchedule`) and **passive walls** (reflective Neumann, $\partial h / \partial n = 0$, via ghost cells mirroring the interior). An absorbing layer (sponge) may line non-piston walls to suppress spurious reflection.

4.3 Time-reversed actuation — wave basis & linear least squares

Because (4.4) is *linear*, the surface field is a superposition of the responses to individual piston impulses. Let $b_k(x, t)$ be the field produced by a unit impulse from piston k at $t = 0$ (its *basis wave*, pre-computed by one forward sim per piston). An arbitrary actuation, where piston k emits amplitude $a_k(\tau)$ at time τ , produces at the focal time T the convolution:

$$h(x, T) = \sum_k \sum_\tau a_k(\tau) \cdot b_k(x, T-\tau) \quad (4.9)$$

Flatten the spatial samples into a row index and the (piston, emission-time) pairs into a column index. Then (4.9) is a single matrix equation in the **wave-basis matrix** $M \in \mathbb{R}^{N \times PT}$ (rows = surface samples, columns = piston-time DOFs), where column (k, τ) holds the flattened field $b_k(\cdot, T-\tau)$:

$$M a = h_t \quad (4.10)$$

The system is generally rectangular and ill-conditioned, so it is solved as a **Tikhonov-regularised least-squares** problem — minimise reconstruction error while penalising physically implausible (high-energy) actuations:

$$a^* = \arg \min_a \| M a - h_t \|^2 + \lambda \| a \|^2 \quad (4.11)$$

whose closed-form solution is the normal equations $(M^T M + \lambda I) a^* = M^T h_t$. The decisive optimisation: M depends only on tank geometry, not on the sketch, so the regularised pseudoinverse is **pre-computed once**:

$$M^+ = (M^T M + \lambda I)^{-1} M^T \implies a^* = M^+ h_t \quad (4.12)$$

so each sketch reduces to *one matrix–vector product* — milliseconds. The connection to the outline's "time-reversal" intuition is exact and worth stating: dropping h_t into the solver and running it backward, then recording boundary energy, computes precisely $M^T h_t$ — the **adjoint**, i.e. the matched filter. Time-reversal is thus the first iteration of the solution; the $(M^T M + \lambda I)^{-1}$ factor is the correction that turns the adjoint into the true least-squares inverse. For a nearly lossless, symmetric tank the two almost coincide, which is why raw time-reversal already produces a recognisable image.

5 Data Constraints & Acceleration Strategies

5.1 Fluid-grid memory layout — Structure-of-Arrays

The stencil update (4.7) is memory-bandwidth-bound, not compute-bound: each cell does a handful of FLOPs but touches five neighbours. Layout therefore dominates performance. Two options:

- **Array-of-Structures (AoS):** `struct Cell { float h, h_prev, v; }; Cell grid[N];` — a cell's fields are contiguous, but the Laplacian reads *only* `h` across neighbours, so every 64-byte cache line drags in unused `h_prev` / `v` bytes. Effective bandwidth utilisation $\approx 1/3$.
- **Structure-of-Arrays (SoA):** `float h[N]; float h_prev[N];` — the same field is contiguous across cells. A cache line now holds 16 consecutive `h` values, all used by neighbouring stencil evaluations, and the row-wise access pattern is trivially auto-vectorised (SSE/AVX/NEON) into SIMD loads.

SoA is mandated for M5. Concretely: two SoA height arrays in a ping-pong pair (read h^n , h^{n-1} ; write h^{n+1} ; swap pointers), each 64-byte aligned, padded with a one-cell **halo/ghost border** so boundary cells run the same branch-free stencil as the interior. Tile the row loop to the L1/L2 footprint if N exceeds cache. This layout also transfers directly to the GPU as a linear R32F texture, so the same buffer feeds M6 without repacking.

Metric	AoS	SoA (chosen)
Cache-line utilisation (Laplacian)	$\sim 33\%$ (2 of 3 fields wasted)	$\sim 100\%$
Auto-vectorisation	Blocked by strided access	Contiguous $\rightarrow$ clean SIMD
GPU upload	Requires deinterleave	Direct as R32F
Ping-pong swap	Copy or index juggling	Pointer swap, $O(1)$

5.2 Caustic raytracing on the GPU — thread mapping & race-free accumulation

The render pass (M6) casts one refracted ray per surface cell and accumulates its light energy where it strikes the floor. The structural difficulty is that *many rays legitimately land on the same floor texel* — that convergence **is** the bright caustic — so the write pattern is a scatter with unavoidable collisions.

Thread $\rightarrow$ cell mapping

Dispatch a 2D grid of `16 $\times$ 16` workgroups; each invocation owns one surface cell $(i,j) = \text{gl_GlobalInvocationID.xy}$. The thread reads the surface normal at its cell, refracts the incoming (vertical) light ray via Snell's law, intersects the refracted ray with the floor plane, and deposits the cell's carried energy into the hit texel. This is the classic *caustic-mapping* / *photon-splatting* formulation — a scatter, not a gather.

The accumulation hazard, and two race-free structures

Because writes collide, a naïve `floor[hit] += energy` is a data race. Two correct approaches:

- **(A) Compute-shader scatter with atomics.** Accumulate into an `R32UI` image; each thread encodes its energy as fixed-point and issues `imageAtomicAdd`. Atomics serialise colliding writes at the memory subsystem, so the sum is exact. Cost: contention in hot texels (bright spots) where thousands of rays converge. Floating-point image atomics are not universally supported, hence the fixed-point `uint` encoding, resolved back to irradiance in a later tone pass.

- **(B) Rasterised additive-blend splatting.** Instead of a compute scatter, render one point/quad primitive per surface cell, positioned at its refracted floor hit, into a float render target with fixed-function *additive blending* enabled. The ROP hardware serialises per-fragment blends, so accumulation is race-free *without* explicit atomics and typically outperforms (A) under heavy convergence.

Gather is the wrong tool here. A gather would assign one thread per floor texel and ask "which surface cells refract onto me?" — but that inverse map is exactly what the simulation does not know without first tracing forward. Scatter (A/B) is therefore the natural structure; the choice between them is a portability-vs-throughput trade (atomics are simpler and compute-only; additive blending is faster but couples M6 to the raster pipeline).

Aspect	(A) Atomic scatter	(B) Additive splat
Race handling	<code>imageAtomicAdd</code> on R32UI	Fixed-function ROP blend
Precision	Exact (fixed-point)	Float blend (order-independent add)
Hot-spot cost	High atomic contention	Amortised in blend units
Pipeline coupling	Pure compute	Needs rasteriser + blend state
Portability	Wide (uint atomics)	Wide, but state-heavy

POST. Either path leaves raw accumulated energy in `floor_accum`. A final full-screen tone pass divides by ray count / exposure and maps to displayable irradiance (R16F), applying optional bloom to sell the high-contrast focal instant.

6 Wave-Physics Modelling Constraints

The real-time budget is affordable only because every heavy step above is linear, and linearity itself is a physical modelling choice with a validity envelope. This section states that envelope.

Small-amplitude / linear superposition. For surface displacement small relative to depth ($h \ll H$) and gentle slopes ($|\nabla h| \ll 1$), the shallow-water equations linearise to (4.4) and the response becomes additive: the field of many piston hits is the sum of individual basis waves (§4.3). This is the single assumption that makes both the inverse solve (4.10) and the pre-computed basis matrix valid. Violating it (large strokes, steep targets) reintroduces nonlinear terms and breaks the superposition the whole pipeline rests on — hence the band-limiting in M2.

Dispersion and dissipation. The damping term $-\gamma \partial h / \partial t$ in (4.4) models viscosity: high spatial frequencies decay fastest, so the reconstructed image loses its finest detail first. This sets a hard resolution ceiling on I and motivates simulating a low-viscosity working fluid (see §9).

Transient focus. Constructive interference reconstructs h_t only at the instant T ; before and after, the surface is an unfocused superposition. The caustic image therefore exists for a fraction of a second by design — addressed by Pulse Mode (§9).

7 Pseudocode Blueprints

High-level, language-agnostic (C++-flavoured) blueprints for the three load-bearing routines. Types reference the §3.2 contracts.

7.1 Main simulation loop — per-frame update (M5 + M7)

```
void simulation_step(FluidState& fs, PistonSchedule& sched,
                    SoAGrid& h_curr, SoAGrid& h_prev, int step) {
    const float dx = fs.cell_size;
    const float c2dt2 = C*C * DT*DT;          //  $c^2 * dt^2$ 
    const float damp = 0.5f * GAMMA * DT;      //  $1/2 \text{ gamma } dt$ 
    const float inv = 1.0f / (1.0f + damp);     //  $1 / (1 + 1/2 \text{ gamma } dt)$ 

    // (1) Interior update: explicit leapfrog, eqn (4.7). SoA + halo => branch-free.
    for (int j = 1; j < H-1; ++j)
        for (int i = 1; i < W-1; ++i) {
            int k = j*W + i;
            float lap = (h_curr[k+1] + h_curr[k-1]
                        + h_curr[k+W] + h_curr[k-W]
                        - 4.0f*h_curr[k]) / (dx*dx);
            h_next[k] = inv * ( 2.0f*h_curr[k]
                              + (damp - 1.0f)*h_prev[k]
                              + c2dt2 * lap );
        }

    // (2) Boundary forcing: piston cells are Dirichlet-driven from the schedule.
    for (int p = 0; p < sched.num_pistons; ++p) {
        float amp = sched.a[p * sched.num_steps + step];
        apply_piston_dirichlet(h_next, piston_cells[p], amp);
    }
    apply_reflective_walls(h_next);           // ghost-cell mirror, Neumann

    // (3) Advance the ping-pong: h_prev <- h_curr <- h_next (pointer swap, O(1)).
    swap(h_prev, h_curr);
    swap(h_curr, h_next);

    // (4) Publish to GPU: upload height, derive normals for the render pass.
    upload_R32F(fs.height, h_curr);
    dispatch_normal_pass(fs.height, fs.normal, dx);
}
```

7.2 Time-reversed boundary recording (M4, adjoint path)

```
// Computes  $a = M^T h_t$  by running the wave solver BACKWARD from the target
// surface and logging the energy that reaches each piston edge each step.
PistonSchedule record_reverse_actuation(TargetHeightmap& tgt) {
    SoAGrid h_curr = embed_target(tgt.h_t); // seed field = target surface
    SoAGrid h_prev = h_curr; // zero initial velocity
    PistonSchedule sched = alloc(NUM_PISTONS, T_REVERSE);

    // Reverse time == same explicit scheme with dt -> -dt. For the symmetric
    // wave operator this is just the forward stencil run on a reversed clock.
    for (int n = 0; n < T_REVERSE; ++n) {
        wave_step_reverse(h_curr, h_prev); // eqn (4.7), reversed

        // Sample the wave energy incident on each piston boundary segment
        // and store it as that piston's required amplitude at this step.
        for (int p = 0; p < NUM_PISTONS; ++p) {
            float e = sample_boundary_flux(h_curr, piston_cells[p]);
            sched.a[p * T_REVERSE + (T_REVERSE-1 - n)] = e; // flip time index
        }
    }

    // OPTIONAL exact solve: replace the adjoint with the regularised inverse,
    //  $a^* = M^+ h_t$  (eqn 4.12), using the pre-factored pseudoinverse.
    if (USE_LEAST_SQUARES)
        sched.a = apply_precomputed_pinv(Mplus, tgt.h_t);

    return sched;
}
```

7.3 GPU compute shader — light refraction core (M6, GLSL-style)

```
// One invocation per surface cell. Scatter path (A): atomic accumulation.
layout(local_size_x = 16, local_size_y = 16) in;

layout(r32f)   uniform image2D  uHeight;           // surface h^n
layout(r32ui)  uniform uimage2D uFloorAccum;      // fixed-point irradiance
uniform float  uDx, uFocalDist, uNrel;           // dx, d, n2/n1

void main() {
    ivec2 c = ivec2(gl_GlobalInvocationID.xy);
    if (c.x < 1 || c.y < 1 || c.x >= W-1 || c.y >= H-1) return;

    // (1) Surface normal via central differences on the heightmap.
    float hx = imageLoad(uHeight, c + ivec2(1,0)).r
               - imageLoad(uHeight, c - ivec2(1,0)).r;
    float hy = imageLoad(uHeight, c + ivec2(0,1)).r
               - imageLoad(uHeight, c - ivec2(0,1)).r;
    vec3 n = normalize(vec3(-hx/(2.0*uDx), -hy/(2.0*uDx), 1.0));

    // (2) Refract the vertical incoming ray through the interface (Snell).
    vec3 incident = vec3(0.0, 0.0, -1.0);
    vec3 refr = refract(incident, n, 1.0/uNrel);
    if (refr == vec3(0.0)) return;           // total internal reflection

    // (3) Intersect refracted ray with the floor plane z = -d.
    float t = uFocalDist / max(-refr.z, 1e-4);
    vec2 hit = vec2(c) * uDx + refr.xy * t;

    // (4) Deposit this cell's energy into the floor texel (race-free add).
    ivec2 tx = world_to_floor_texel(hit);
    if (in_bounds(tx)) {
        uint q = encode_fixed(CELL_ENERGY);    // float -> fixed-point
        imageAtomicAdd(uFloorAccum, tx, q);     // serialised, exact
    }
}
```

8 Development Milestones & Implementation Plan

Phase	Description & key deliverables	Core technologies
1 · Engine Foundation	Real-time damped-wave FDM (M5) with verified CFL stability, reflective/absorbing boundaries, and the forward refraction shader (M6). Acceptance: a manually poked ripple reflects correctly and casts a moving caustic.	Vulkan / OpenGL 4.5 / WebGPU compute; C++17 or Rust; SoA grid.
2 · Mathematical Core	Poisson inverse-caustic solver (M3) with optional Monge–Ampère refine; time-reversed / least-squares actuation mapper (M4). Acceptance: piston timings logged and validated against a synthetic target heightmap.	FFT/DCT or multigrid Poisson; Eigen / SciPy for the pseudoinverse; golden-image tests.
3 · Piston Automation	Close the loop: feed <code>PistonSchedule</code> into the forward sim; calibrate γ , Δt , and λ so the surface converges cleanly at T . Acceptance: recognisable reconstruction of a test glyph.	M7 state machine; delta-time playback clock; regularisation tuning.
4 · User Interface	2D drawing canvas (M1) wired to the pipeline; orbit camera; Pulse-Mode toggle and exposure control. Acceptance: sketch-to-caustic round trip under the frame budget.	React + WebGPU (web) or Dear ImGui (native).

9 Technical Challenges & Risk Mitigation

Challenge	Failure mode	Mitigation
Wave dissipation	Viscosity decays high-frequency ripples fastest; fine image detail is lost before focus.	Band-limit the canvas (M2 blur) to formable frequencies; simulate a low-viscosity fluid (e.g. alcohol, smaller γ).
Transient focus	The reconstructed image exists for only a fraction of a second before dispersing.	Pulse Mode: M7 loops the schedule, re-emitting synchronised wavefronts so the image pulses steadily.
OT overhead	Monge–Ampère Newton solves cost 10^2 – 10^3 ms — far over budget for interactive editing.	Default to the $O(N \log N)$ Poisson solve (§4.1.2); down-sample the solver grid; reserve OT for an explicit refine.
Numerical instability	Explicit leapfrog diverges if the timestep violates CFL (4.8).	Derive Δt from $\Delta x, c$ at $0.9\times$ the CFL bound; assert the ratio at init.
Atomic contention	Bright caustic texels receive thousands of concurrent atomic adds, stalling the scatter.	Prefer additive-blend splatting (path B) under heavy convergence; otherwise fixed-point R32UI atomics with a resolve pass.
Reconstruction accuracy	Adjoint (time-reversal) alone leaves ghosting from the omitted $(M^T M)^{-1}$ term.	Apply the pre-factored regularised pseudoinverse (4.12) when quality matters; tune λ for the energy/fidelity trade.

A Nomenclature

Symbol	Meaning
$I(x, y), I^-$	Target light-density map and its uniform source (mean) level.
$h(x, y, t), h_t$	Surface height field; target heightmap at the focal instant.
ϕ, u	Optimal-transport potential ($\nabla\phi$ = transport map); paraxial displacement potential.
c, γ	Shallow-water wave speed $\sqrt{gH}$; linear damping coefficient.
$\Delta x, \Delta t$	Grid spacing and timestep of the FDM discretisation.
$L[\cdot]$	Discrete five-point Laplacian operator (4.5).
M, M^+, a	Wave-basis matrix, its regularised pseudoinverse, and the piston-amplitude vector.
b_k	Basis wave: surface response to a unit impulse from piston k .
λ	Tikhonov regularisation weight in the least-squares actuation solve (4.11).
d, n_{rel}	Surface-to-floor focal distance; relative refractive index n_2/n_1 .
N, N', P, T	Sim grid cells, solver grid cells, piston count, focal-time step index.

Dynamic Fluid Caustics — Technical Specification, Rev. 2.0. Prepared as a refinement of the source project outline. Simulation regime: linearised shallow-water surface waves. All algorithmic cost figures are order-of-magnitude at 256^2 solver resolution on commodity desktop GPUs.